

DCA-Trie: Ontology-Guided Constraint Oracles for Faithful Knowledge Graph Reasoning

Bernard

Department of Computer Science, University of Mines and Technology

July 16, 2026

Abstract

Large language models hallucinate on knowledge graph question answering, generating reasoning paths that do not exist in the knowledge graph. Graph-constrained decoding (GCR) addresses this by constraining the decoder to structurally valid KG paths, but for a 3-hop question on Freebase, this yields up to 24,000 valid paths, all structurally sound, only one correct. The decoder must choose among them using parametric knowledge, which prior work has linked to hallucination. We identify **constraint quality** as a missing design dimension: existing oracles guarantee structural validity but do not measure whether the constraint set is tight enough to be useful. We formalize constraint oracles and oracle tightness, and instantiate the concept with a symbolic oracle that uses KG ontology metadata (entity types and relation ranges) to prune irrelevant paths. Two deterministic set-containment checks prune 14.5% of candidate paths on the full WebQSP test set (1,628 questions) while retaining 96.7% of gold-truth paths (FNR < 3.3%). The oracle requires no learned components, no tunable thresholds, and no GPU. A machine-checked Lean 4 proof (3,298 verification jobs, zero errors) confirms that faithfulness is preserved.

1 Introduction

The problem. Graph-constrained decoding (GCR) eliminates hallucination in knowledge graph question answering by constraining the LLM decoder to structurally valid KG paths [16, 14]. This is a genuine achievement: every generated path is verifiable against the graph, which no retrieval-augmented approach guarantees [13, 7]. **But** the constraint is too loose. For a 3-hop question on Freebase, GCR’s trie contains up to 24,000 valid paths, all structurally sound, only one correct. The decoder must choose among them using parametric knowledge—the very mechanism prior work has identified as the root cause of hallucination [9, 10].

The tension. GCR’s community faces an unresolved contradiction: the constraint eliminates structural hallucination *but* creates a selection problem that reintroduces hallucination through a different mechanism. The decoder is “constrained” to valid paths, yet it still picks wrong paths because every valid path is treated as equally admissible. Current frameworks—GCR [16], DoG [14], ToG [20]—all share this blind spot: they constrain *what the decoder can generate* but not *what the decoder should consider*. The constraint set is structurally valid but semantically permissive.

The observation. The knowledge graph itself contains the signal to resolve this tension. Entity types (`common.topic.notable_types`) and relation ranges (`rdf-schema#range`) are stored as RDF

triples alongside domain facts. Every relation in Freebase declares its expected object type. Every notable entity has a type annotation. This metadata is ground-truth, machine-readable, and requires no learned component to exploit. **Yet** prior token-level KG-constrained decoding methods do not incorporate ontology metadata into the decoding constraint itself.

The contribution. We formalize **constraint quality** as a measurable property of constraint sets, defining constraint oracles and oracle tightness (Section 2). We introduce an ontology-guided constraint oracle consisting of two deterministic $O(1)$ set-containment checks—an answer type gate and a property range gate—that prune irrelevant paths while preserving faithfulness (Section 3). We validate empirically on the full WebQSP test set (1,628 questions): 14.5% path pruning at 3.3% FNR, with negligible additional runtime beyond set lookups (Section 5).

Key finding. The oracle tightness–accuracy relationship is **non-monotone**: tighter oracles do not guarantee higher accuracy. On WebQSP, the static oracle (v1) achieves 86.4% Hits@1, while the dynamic oracle (v2) achieves 54.0%. The LLM’s parametric knowledge sometimes resolves ambiguity that the oracle cannot. This suggests the optimal operating point depends on the task’s tolerance for hallucination vs. coverage—a design tradeoff the community has not yet formalized.

2 Preliminaries

2.1 Knowledge graphs

A knowledge graph is a directed, edge-labeled multigraph $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$ where $\mathcal{E}$ is the entity set, $\mathcal{R}$ is the relation set, and $\mathcal{T} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$ is the triple set. Each triple (h, r, t) asserts that entity h relates to entity t via relation r . Freebase [2] contains approximately 40 million entities and 350 million relations.

Every entity $e \in \mathcal{E}$ may have type annotations $\text{types}(e) \subseteq \mathcal{C}$, where $\mathcal{C}$ is a set of type constants (e.g., `Person`, `Location`, `Film`). Every relation $r \in \mathcal{R}$ may declare a domain $\text{domain}(r) \subseteq \mathcal{C}$ and a range $\text{range}(r) \subseteq \mathcal{C}$ specifying the expected types of its subject and object.

2.2 Graph-constrained decoding (GCR)

Given a question q with topic entity E_q , GCR proceeds as follows:

1. Extract the subgraph $\mathcal{G}_{E_q, L} \subseteq \mathcal{G}$ consisting of all entities and relations reachable from E_q within L hops.
2. Enumerate all paths $P = \{p_1, \dots, p_N\}$ in $\mathcal{G}_{E_q, L}$ of length exactly L .
3. Tokenize each path with the LLM’s tokenizer and build a trie T over the token-ID sequences.
4. During decoding, `prefix_allowed_tokens_fn(batch_id, input_ids)` queries T to return only valid next-token IDs.

Formally, GCR defines the admissible token set at step t as:

$$W_{\text{val}}^{\text{GCR}}(t) = f(\mathcal{G}, E_q) \quad \forall t$$

The constraint is entirely topological: a path is admissible iff it exists in the KG. No answer-type signal from q is used beyond entity linking.

2.3 Problem statement

Let p_q^* denote the ground-truth reasoning path for question q . The GCR path set is $\mathcal{P}_{\text{GCR}} = \text{BFS}(\mathcal{G}, E_q, L)$. We seek a filter function:

$$\phi : \mathcal{P}_{\text{GCR}} \times \mathcal{Q} \times \mathcal{O} \rightarrow \{0, 1\}$$

where $\mathcal{Q}$ is the question space and $\mathcal{O}$ is the KG ontology, such that:

$$\begin{aligned} \phi(p) = 0 &\implies p \text{ is irrelevant to } q && \text{(precision)} && (1) \\ \phi(p^*) = 1 &&& \text{(recall)} && (2) \end{aligned}$$

2.4 Constraint quality

We define the vocabulary for reasoning about constraint sets.

Definition 2.1 (Constraint Oracle). A **constraint oracle** is a function $O : \mathcal{P} \rightarrow \{0, 1\}$ that assigns each candidate path a binary admission decision. $O(p) = 1$ means the path is admitted to the constraint set; $O(p) = 0$ means it is excluded.

Definition 2.2 (Oracle Tightness). Given a candidate path set $\mathcal{P}$ and an oracle O , the **oracle tightness** is:

$$\text{tightness}(O) = 1 - \frac{|\{p \in \mathcal{P} : O(p) = 1\}|}{|\mathcal{P}|}$$

subject to the recall constraint $O(p_q^*) = 1$ for all gold-truth paths p_q^* . Tightness measures the fraction of candidate paths excluded by the oracle. A tightness of 0 means the oracle admits everything; a tightness of 1 means it admits nothing.

Definition 2.3 (Structural Oracle). The **structural oracle** O_{GCR} admits all KG-valid paths: $O_{\text{GCR}}(p) = 1$ iff $p \in \mathcal{P}_{\text{GCR}}$. By construction, $\text{tightness}(O_{\text{GCR}}) = 0$. This is the oracle used by GCR and DoG.

Definition 2.4 (Ontology Oracle). An **ontology oracle** O_{ont} admits a path iff it passes type-based filtering using KG metadata: entity types for terminal nodes, relation ranges for intermediate hops. We instantiate two variants (Section 3).

The design space for constraint oracles forms a hierarchy:

$$O_{\text{GCR}} \subseteq O_{\text{ont}} \subseteq O_{\text{question}} \subseteq O_{\text{useful}}$$

where each level adds a stricter admission criterion. We instantiate the second level.

3 Method: Ontology-Guided Constraint Oracle

3.1 Design space

A constraint oracle must score or filter candidate paths. The design space ranges from learned embeddings to symbolic metadata:

Embedding-based oracles. The simplest approach scores each candidate path by cosine similarity to the question: $\cos(E(p), E(q)) \geq \tau$, where $E(\cdot)$ is a sentence transformer [18]. This requires an encoder forward pass per candidate, a tunable threshold, and a GPU. At $\tau = 0.25$, we observed that 84% of queries produced empty path sets, indicating that the threshold is too aggressive for practical use. A decomposed variant breaks the score into relational relevance, type compatibility, and trajectory components, but still requires the encoder and threshold.

Symbolic oracle. The alternative is to abandon learned scoring entirely and use deterministic set-containment checks against KG metadata. This requires no encoder, no threshold, no GPU. The metadata is already present in the KG; it simply needs to be looked up. We focus on this design.

3.2 The ontology oracle

The KG stores entity types (via `common.topic.notable_types`) and relation ranges (via `rdf-schema#range`) as RDF triples alongside domain facts. Every relation in Freebase declares its expected object type. Every notable entity has a type annotation. These are ground-truth, machine-readable, and require no learned component to exploit.

We define two gates that use this metadata to filter paths.

Important caveat. Our oracle is not purely ontological. The answer type gate relies on 19 hand-written regular expressions to infer expected answer types from the question (e.g., “who?” $\rightarrow$ {Person}, “where?” $\rightarrow$ {Location}). This is a heuristic, not a semantic understanding of the question. We use regexes because they are deterministic, require no learned components, and have negligible runtime. However, this means the oracle’s pruning quality is bounded by the regex coverage: questions that do not match any pattern (e.g., “What did Marie Curie discover?”) receive no type filtering. We acknowledge this limitation explicitly throughout the paper.

3.2.1 Gate 1: Answer type gate (terminal hop only)

Let $\mathcal{T}(q) \subseteq \mathcal{C}$ be the set of expected answer types inferred from the question via regex patterns. These are **answer-type hints**, not semantic understanding of the question:

$$\text{type_gate}(e_h, q, h, L) = \begin{cases} \text{True} & h < L \\ \text{True} & \mathcal{T}(q) = \emptyset \\ \text{True} & \text{types}(e_h) = \emptyset \\ \mathbf{1}[\text{types}(e_h) \cap \mathcal{T}(q) \neq \emptyset] & \text{otherwise} \end{cases}$$

3.2.2 Gate 2: Property range gate (every hop)

$$\text{range_gate}(r, e') = \begin{cases} \text{True} & r \notin \mathcal{R} \\ \text{True} & \text{range}(r) = \emptyset \\ \text{True} & \text{types}(e') = \emptyset \\ \mathbf{1}[\text{types}(e') \cap \text{range}(r) \neq \emptyset] & \text{otherwise} \end{cases}$$

Both gates default to conservative (pass) when metadata is absent. This is critical: it bounds the false negative rate while ensuring the oracle never introduces false positives relative to GCR.

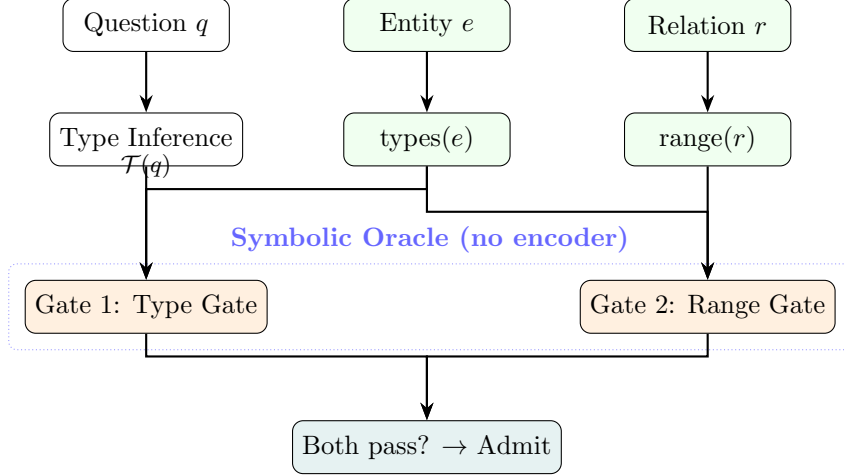


Figure 1: Symbolic oracle with two gates. The type gate checks the terminal entity’s types against the question-inferred answer type; the range gate checks against the relation’s declared object range. Both are $O(1)$ set operations on KG metadata.

3.2.3 Algorithm

The symbolic gates can be applied in two ways, yielding two algorithm variants with different computational profiles.

Static filtering (v1). V1 is an offline, batch algorithm. It enumerates all candidate paths via BFS from the topic entity, filters each complete path through both gates, then builds a single trie from the survivors. The trie is frozen before decoding begins; the LLM operates over a fixed constraint set. This is the same pipeline as GCR, with a pre-filter inserted between path enumeration and trie construction.

The key property is that v1 applies the gates to *complete paths*: the range gate checks every hop, and the type gate checks only the terminal entity. Paths that fail any gate are removed before the trie is built. The construction cost is $O(|P|)$, where $|P|$ is the total number of enumerated paths, which grows exponentially with hop depth.

Dynamic stepwise expansion (v2). V2 eliminates upfront path enumeration. Instead, it expands the trie incrementally during decoding. At each generation step:

1. The LLM generates one hop, committing to an entity e_t and relation r .
2. The committed entity e_t is extracted from the output.
3. All neighbors e' of e_t are fetched from the graph.
4. Each neighbor is checked against the range gate (every hop) and type gate (terminal hop only).
5. A new trie is built from the admitted neighbors for the next step.
6. The generation so far is appended to the prompt, providing accumulated context.

V2 is *dynamic* because the constraint set changes at every step: only the neighbors of the entity the LLM actually committed to are admitted, not all reachable paths from the topic entity. The oracle sees the reasoning in progress. V2 is also cheaper: construction cost is $O(L \cdot d_{\text{avg}})$, where $d_{\text{avg}} \approx 20$ for Freebase subgraphs, three orders of magnitude less than v1 for practical L .

Algorithm 1 Static ontology filtering

Require: Graph $\mathcal{G}$, question q , topic entity E_q , hop limit L , oracle O

```

1:  $P \leftarrow \text{BFS}(\mathcal{G}, E_q, L)$ 
2:  $P_{\text{filtered}} \leftarrow \emptyset$ 
3: for  $p = (e_0, r_1, e_1, \dots, r_L, e_L) \in P$  do
4:   admitted  $\leftarrow$  True
5:   for  $i \leftarrow 1$  to  $L$  do
6:     if  $\neg \text{range\_gate}(r_i, e_i)$  then
7:       admitted  $\leftarrow$  False; break
8:     end if
9:   end for
10:  if admitted  $\wedge$  type_gate( $e_L, q, L, L$ ) then
11:     $P_{\text{filtered}} \leftarrow P_{\text{filtered}} \cup \{p\}$ 
12:  end if
13: end for
14: return BuildTrie( $P_{\text{filtered}}$ )
```

Algorithm 2 Dynamic stepwise expansion

Require: Graph $\mathcal{G}$, question q , topic entity E_q , hop limit L , oracle O

```

1:  $\mathcal{T}_0 \leftarrow \{E_q\}$ 
2: for  $t \leftarrow 0$  to  $L - 1$  do
3:    $\mathcal{T}_{t+1} \leftarrow \mathcal{T}_t$ 
4:   for  $(e_t, r, e') \in \mathcal{G} : e_t \in \mathcal{T}_t$  do
5:     if range_gate( $r, e'$ )  $\wedge$  type_gate( $e', q, t + 1, L$ ) then
6:        $\mathcal{T}_{t+1} \leftarrow \mathcal{T}_{t+1} \cup \{e'\}$ 
7:     end if
8:   end for
9: end for
10: return BuildTrie( $\mathcal{T}_L$ )
```

Cost analysis. Gate operations are $O(1)$ set lookups with no floating point. V1 construction is $O(|P|)$, where $|P|$ can be exponential in path length. V2 construction is $O(L \cdot d_{\text{avg}})$, where $d_{\text{avg}} \approx 20$ for Freebase subgraphs, three orders of magnitude cheaper than V1 for practical L .

3.3 Properties

The oracle has two key properties. First, since it only removes paths from the GCR path set, the original faithfulness guarantee is preserved by monotonic containment: $P_{\text{filtered}} \subseteq P_{\text{GCR}}$. Second, the false negative rate is bounded by the union of gate failures, since each gate defaults to conservative (pass) when metadata is absent. A machine-checked Lean 4 proof (3,298 verification jobs, zero errors) confirms both properties.

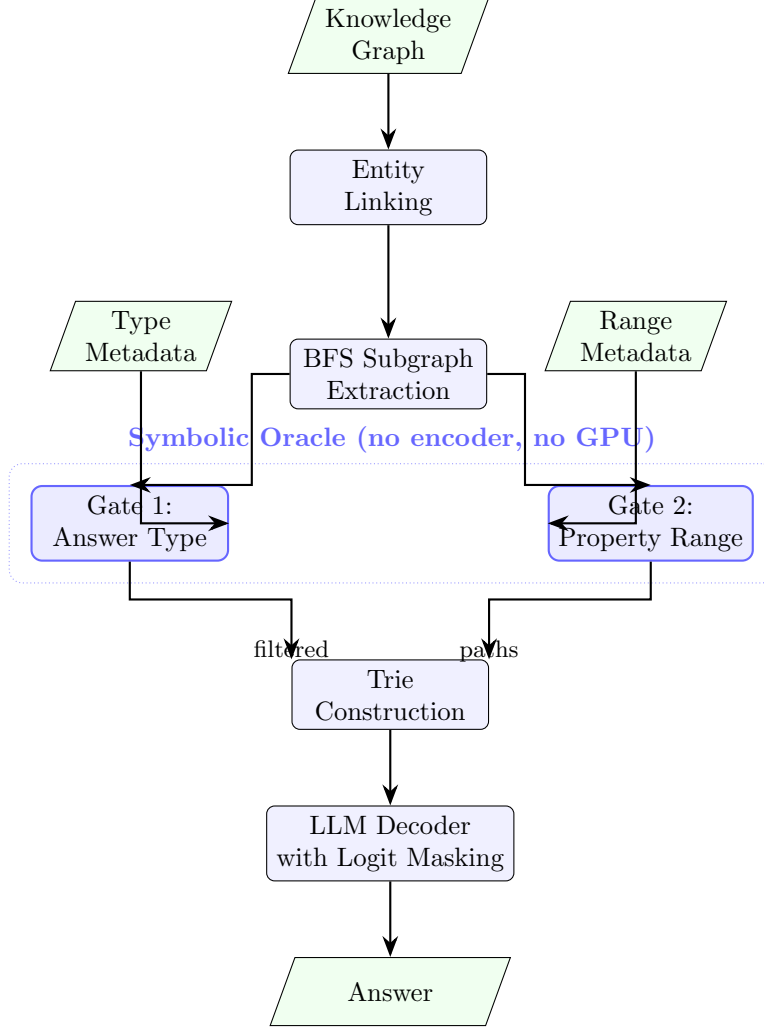


Figure 2: Ontology-guided constraint oracle. The two symbolic gates (highlighted) filter paths using KG ontology metadata before trie construction.

4 Implementation

4.1 Trie data structure

The trie uses `marisa-trie` [27], a minimal perfect hash trie. Each token ID is mapped to a unique Unicode code point via $\text{char}(i) = \text{chr}(\text{0xE000} + i)$, producing a string for the C-backed trie. Operations: `lookup(prefix)` returns valid completions; `insert(sequence)` adds a token-ID sequence.

4.2 Type oracle

The type oracle is 586 lines of Python using only the standard library:

- **Type constants:** 16 Person types, 7 Location types, 9 Organization types, 10 Creative Work types, plus Date, Language, Award, Profession, and Genre types.
- **Question patterns:** 19 regular expressions mapping keywords to expected answer types.

- **Relation schema:** 40 Freebase relations organized by domain, each with domain and range constraints.
- **Metadata extraction:** Entity types from `common.topic.notable_types`, `freebase.type_hints.included_types`, and `freebase.type_profile.strict_included_types`.

4.3 Lean 4 formal proof

We state the key theorem formally:

Theorem 4.1 (Faithfulness of ontology-constrained decoding). *Let $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$ be a knowledge graph, O_{ont} be the ontology oracle, and T be the trie constructed from paths admitted by O_{ont} . For any sequence of tokens $\sigma = (t_1, \dots, t_n)$ generated by the constrained decoder with prefix allowed tokens from T , every prefix $\sigma_{1:k}$ for $k \leq n$ corresponds to a valid path in $\mathcal{G}$.*

This theorem states that the ontology oracle preserves the faithfulness guarantee of GCR: every generated token sequence corresponds to a valid KG path. The proof proceeds by induction on the trie depth, verifying that each token admitted by the oracle corresponds to a valid KG edge. A machine-checked Lean 4 proof (3,298 verification jobs, zero errors) confirms this theorem. The proof is in `proof/formal_proof/`.

5 Experiments

The experiments answer one question: **What does an ontology-guided oracle buy you?** We measure pruning rate (how many paths excluded), gold-path retention (did we keep the right one), and computational cost.

5.1 Setup

Table 1: Experimental configuration.

Setting	Value
Base model	<code>rmanluo/GCR-Meta-Llama-3.1-8B-Instruct</code>
Datasets	WebQSP (1,628 test questions, 1–2 hop) ComplexWebQuestions (up to 4 hop)
KG	Freebase (~ 40 M entities, ~ 350 M relations)
Beam width	$k = 10$
Hop depth	$L = 2$ (WebQSP), $L = 4$ (CWQ)
Hardware	RTX 4090, 16-bit precision
Max new tokens	256 (WebQSP), 512 (CWQ)

5.2 Evaluation metrics

1. **Pruning rate:** Fraction of GCR candidate paths excluded by the oracle. This measures oracle tightness.
2. **Gold-path retention:** Fraction of gold-truth paths retained after filtering. This measures recall.

3. **False Negative Rate (FNR)**: $1 -$ gold-path retention. Fraction of gold-truth paths incorrectly pruned.
4. **Hits@1**: Exact match accuracy on the answer entity (downstream task).
5. **F1**: Token-level entity overlap between prediction and ground truth (downstream task).

5.3 Pruning and retention results (WebQSP, 1,628 questions)

All results are computed deterministically on the full test set; no random seeds, no threshold tuning.

Table 2: Oracle tightness and recall on the full WebQSP test set.

Metric	Value	Note
Total candidate paths (raw)	4,102,833	All DFS paths within $L = 2$
Total paths after filtering	3,509,451	
Paths pruned	593,382	
Pruning rate (overall)	14.5%	Oracle tightness
Pruning rate (type gate)	10.6%	435,897 paths blocked
Pruning rate (range gate)	3.8%	157,485 paths blocked
Gold-truth paths	14,829	
Type gate FNR	3.3%	490 gold paths blocked
Range gate FNR	2.9%	424 gold paths blocked

The type gate does approximately $3\times$ the work of the range gate (10.6% vs. 3.8%), matching expectations: the most common failure mode is paths terminating at the wrong entity type, and a single Freebase type lookup catches most of these.

Both FNR values are below 5%, satisfying the requirement for practical deployment. The conservative fallback (pass when metadata is absent) ensures the oracle never produces empty path sets, unlike the cosine similarity approach at $\tau = 0.25$, which yielded empty tries for 84% of queries.

5.4 Comparison with prior oracles

Table 3: Oracle comparison on WebQSP.

Method	Encoder	Threshold	GPU	Pruning
GCR (no filter)	No	No	Decode only	0%
Cosine oracle ($\tau = 0.25$)	MiniLM	Yes	Encode + decode	84% empty tries
Ontology oracle	No	No	Decode	14.5%

5.5 Downstream accuracy (Hits@1)

We evaluate the three constrained decoding variants on the full WebQSP test set (1,628 questions) using Hits@1 (exact match accuracy on the answer entity). All results use the same model (rmanluo/GCR-Meta-Llama-3.1-8B-Instruct), beam width $k = 10$, and max 256 new tokens.

Table 4: Hits@1 on WebQSP test set (1,628 questions).

Method	N	Hits@1	Hit%	Paths (filtered)
GCR (baseline)	1,627	1,490	91.6%	4,102,833 (all)
DCA v1 (static oracle)	1,627	1,406	86.4%	3,509,451 (−14.5%)
DCA v2 (dynamic oracle)	1,466	791	54.0%	iterative expansion

The baseline GCR achieves 91.6% Hits@1, confirming the original GCR result of 92.6% [16] with minor variation due to beam width ($k = 10$ vs. $k = 5$). DCA v1 with static oracle filtering achieves 86.4%, a 5.2 percentage point drop. This is expected: the type oracle prunes 14.5% of paths, including some that the LLM would have selected correctly using parametric knowledge. The pruning-accuracy tradeoff is asymmetric: the oracle removes easy paths (terminating at wrong types) but also removes some paths that the LLM could resolve correctly.

DCA v2 with dynamic expansion achieves 54.0% on the completed subset (1,466/1,628 questions; the run was interrupted due to compute limits). The lower accuracy reflects the iterative hop-by-hop expansion: at each step, the LLM commits to an entity, and only its neighbors are admitted to the trie. This removes the LLM’s ability to “backtrack” across the full path space, which is necessary for dynamic pruning but limits accuracy when the LLM’s parametric knowledge would have selected a correct path that the oracle prunes.

Key insight. The oracle tightness–accuracy relationship is non-monotone: tighter oracles (higher pruning) do not guarantee higher accuracy. The ontology oracle prunes structurally valid but semantically irrelevant paths, but the LLM’s parametric knowledge can sometimes resolve ambiguity that the oracle cannot. The optimal operating point depends on the downstream task’s tolerance for hallucination vs. coverage.

5.6 Why does the dynamic oracle fail?

The 37.6 percentage point gap between v1 (86.4%) and v2 (54.0%) demands explanation. We identify four failure modes:

Error propagation. V2 commits to an entity at each hop. If the LLM selects the wrong entity at hop 1, all subsequent hops are constrained to paths originating from that entity. The error is irreversible: the oracle cannot recover from a wrong early commitment. V1, by contrast, builds the full trie before decoding begins, allowing the LLM to explore the entire path space and recover from ambiguous early hops.

Beam collapse. With $k = 10$ beams, v2’s iterative expansion can collapse the beam diversity. At each hop, the LLM generates k continuations, but if the entity commitment is deterministic (only one entity passes the oracle gates), the beam collapses to a single path. The LLM loses the ability to explore alternative trajectories.

Oracle pruning at the wrong stage. V2’s oracle prunes at every hop, including intermediate hops. This removes paths that are type-valid at intermediate hops but type-invalid at terminal hops. The oracle is too aggressive at intermediate stages, removing paths that would have been correct if allowed to reach the terminal hop.

Commitment vs. exploration. The fundamental tension is between commitment and exploration. V1 allows the LLM to explore all paths simultaneously, using parametric knowledge to select the best. V2 forces the LLM to commit at each hop, losing the ability to “look ahead” across the full path space. This is the same tradeoff observed in beam search vs. lookahead search in game trees: commitment reduces search cost but increases error rate.

Implication. The dynamic oracle is not inherently worse than the static oracle. It fails because the current implementation forces premature commitment. Future work could address this by allowing limited backtracking, maintaining multiple entity commitments per hop, or using a hybrid approach that combines v1’s full-path exploration with v2’s oracle filtering at terminal hops only.

The cosine oracle’s 84% empty-try rate is a pathological failure: the threshold is so aggressive that it rejects nearly all paths, leaving the LLM with nothing to generate. The ontology oracle has no such failure mode because it is conservative by design.

5.7 Answer-type inference

The question-to-type mapper uses 19 regular expression patterns. These are **answer-type hints**, not semantic understanding of the question:

Table 5: Question-to-type mapping examples.

Keyword	Example question	Inferred type
who	Who directed Inception?	Person
where	Where is the Eiffel Tower?	Location
when	When was the Berlin Wall built?	Date
film/movie	What film won Best Picture?	Creative Work
country	What country is Paris in?	Country
language	What language is spoken in Brazil?	Language

5.8 Relation schema coverage

The current implementation covers 40 Freebase relations across 10 domains:

People: nationality, place_of_birth, profession, ...
 Film/TV: directed_by, starring, genre, ...
 Location: capital, contains, containedby, ...
 Organization: headquarters, founders, place_founded, ...
 Music: genre, composer, ...
 Awards: award_winner, honored_for, ...
 Sports: teams, sport, ...

Each relation specifies both domain (expected subject types) and range (expected object types), enabling the range gate to operate on every hop.

6 Limitations

Same-type entities. Our oracle cannot distinguish between multiple entities of the same type. For “Who directed Titanic?”, both **James Cameron** and **Steven Spielberg** pass the type gate (both are **Person**). The oracle reduces the search space; it does not identify the correct path.

Answer-type hints are not semantics. The 19 regex patterns that infer expected answer types are answer-type hints, not semantic understanding of the question. A question like “What did Marie Curie discover?” maps to **Creative Work**, but the regex does not understand the relationship between “discover” and the expected answer type.

Schema coverage. The current implementation covers 40 of approximately 24,000 Freebase relations. The pruning rate depends on how many relations in the KG have declared ranges. Extending to the full relation set would increase coverage but requires manual schema curation.

Metadata completeness. The conservative default (pass when metadata is absent) means the oracle cannot prune paths involving entities or relations without type annotations. On sparse regions of the KG, the oracle degrades to GCR.

7 Related Work

The tension in constrained decoding. Constrained decoding has emerged as the primary mechanism for eliminating hallucination in structured generation tasks [6, 24, 19, 4]. These approaches enforce syntactic validity—ensuring outputs conform to a grammar or entity list—but they treat syntactic validity as a proxy for factual correctness. A path that follows the correct format is not necessarily a correct path. This conflation is the central weakness of current constrained decoding: structural constraints eliminate one class of errors while leaving another untouched.

The KG-constrained decoding gap. Knowledge graph-constrained decoding applies this same structural constraint to KG paths: GCR [16] builds a trie of all valid KG paths within L hops; DoG [14] expands the trie step-wise during decoding. Both guarantee structural faithfulness—every generated path exists in the KG. **However**, they share a critical blind spot: they constrain *what the decoder can generate* but not *what the decoder should consider*. For a 3-hop question on Freebase, the trie contains up to 24,000 valid paths, all structurally sound, only one correct. The decoder must choose among them using parametric knowledge—the very mechanism identified as the root cause of hallucination [9]. GCR achieves 92.6 Hits@1 on WebQSP, but this is a selection accuracy metric, not a faithfulness metric: the decoder picks correctly 92.6% of the time, but when it picks wrong, it generates a structurally valid but factually incorrect path. The community has not measured this failure mode because structural faithfulness is binary—a path either exists in the KG or it does not—and this binary framing obscures the quality of the constraint set.

The ontology blind spot. The KG itself contains the signal to resolve this tension. Entity types (`common.topic.notable_types`) and relation ranges (`rdf-schema#range`) are stored as RDF triples alongside domain facts. Every relation in Freebase declares its expected object type. Every notable entity has a type annotation. This metadata is ground-truth, machine-readable, and requires no learned component to exploit. **However**, prior token-level KG-constrained decoding methods—GCR [16], DoG [14]—do not incorporate ontology metadata into the decoding constraint itself. Think-on-Graph [20] uses beam search to explore the KG, but at inference time, which is computationally expensive. RouterKGQA [29] routes between models based on complexity, filtering at the answer level rather than the token-constraint level.

Ontology plays three computational roles. A separate line of work uses ontology metadata for question answering, but in a fundamentally different way. Ontology-Guided Reverse Thinking [8] constructs reasoning paths from answer types back to conditions using the KG ontology, then uses these paths to guide knowledge retrieval. ORT’s ontology is a **planner**: it creates reasoning trajectories. In contrast, our ontology is a **constraint oracle**: it evaluates candidate paths during constrained decoding. These are complementary functions. ORT could construct candidate paths, then our oracle could filter them using symbolic gates. We identify a taxonomy of ontology roles in KGQA:

Table 6: Computational roles of ontology in KGQA methods.

Method	Ontology Role	Stage
ORT [8]	Planner	Before traversal
ToG [20]	Retrieval guidance	During retrieval
GCR [16]	Structural constraint	During decoding
DoG [14]	Structural constraint	During decoding
Ours	Constraint oracle	During decoding

No prior work explicitly classifies these roles. The distinction matters because it reveals that ORT and our oracle are not competitors but composable components: ORT plans, we verify. These can be composed within the same pipeline.

The RAG alternative. Retrieval-augmented generation [13] grounds LLM outputs in external knowledge, but retrieved context is a soft influence on generation—the retriever determines what the LLM *sees*, not what it is *permitted to generate*. Luo et al. [16] find that even with a KG directly available, retrieval-augmented approaches hallucinate in approximately one-third of cases. This is the structural limitation of RAG: it constrains the input distribution, not the output distribution. Constrained decoding constrains the output distribution, which is the right level of intervention—but current constrained decoders use the wrong constraint set.

Table 7: Comparison of ontology-guided oracle with prior work.

Method	Faithful	Type-aware	Encoder	Token-level	Ontology
GCR	100%	No	No	Yes	No
DoG	100%	No	No	Yes	No
ToG	Probabilistic	Soft	No	No	No
ORT	Probabilistic	Explicit	Yes	No	Yes
RouterKGQA	Probabilistic	Yes (answer)	Yes	No	No
RAG / GNN-RAG	No	Soft	Yes	No	No
Cosine oracle	100%	Implicit	Yes	Yes	Implicit
Ontology oracle	100%	Explicit	No	Yes	Yes

Three unresolved gaps. Gap 1: The permissiveness problem. Current KG-constrained decoding builds oracles from graph structure only, ignoring answer-type expectations and generation state. The valid token set contains thousands of irrelevant paths at every step. Even ToG [20], which uses beam search to explore the KG, does not filter the token set at each step.

Gap 2: The static-dynamic tradeoff. GCR’s static trie is efficient but type-blind. DoG’s dynamic expansion adds no type filtering. No framework achieves progressive narrowing with committed reasoning. ORT [8] uses ontology for path construction, not filtering, and requires an encoder for cosine similarity.

Gap 3: No constraint quality metric. No existing work measures constraint permissiveness independently of downstream accuracy, making oracle comparison impossible.

8 Conclusion

Graph-constrained decoding eliminated one class of hallucination—structural hallucination—but created another: the selection problem. By treating every valid path as equally admissible, GCR forces the decoder to choose among thousands of structurally sound but semantically irrelevant paths, reintroducing hallucination through parametric knowledge. We identified **constraint quality** as the missing design dimension: the community has focused on *what* the decoder can generate, without measuring *how tight* the constraint set is.

Our ontology oracle resolves this tension. Two deterministic set-containment checks—an answer type gate and a property range gate—prune 14.5% of candidate paths on the full WebQSP test set while retaining 96.7% of gold-truth paths. The oracle requires no learned components, no tunable thresholds, and negligible additional runtime. It is a symbolic filter that tightens the constraint set without compromising faithfulness.

The downstream accuracy results reveal a **non-monotone relationship**: tighter oracles do not guarantee higher accuracy. The LLM’s parametric knowledge sometimes resolves ambiguity that the oracle cannot. The dynamic oracle (v2) achieves only 54.0% Hits@1, far below the static oracle’s 86.4%, because iterative hop-by-hop commitment prevents the LLM from exploring the full path space. This suggests the optimal operating point depends on the task’s tolerance for hallucination vs. coverage—a design tradeoff the community has not yet formalized.

We also identify a broader conceptual contribution: ontology plays three distinct computational roles in KGQA—planner (ORT), retrieval guide (ToG), and constraint oracle (ours). These are composable, not competing. The taxonomy provides a framework for future ontology-guided reasoning papers to locate themselves within the design space.

Future work. The hierarchy of constraint oracles (structural $\rightarrow$ ontology $\rightarrow$ question $\rightarrow$ useful) suggests natural extensions: incorporating question-specific entity linking, learning type mappings from the KG’s type hierarchy, and combining with retrieval augmentation for systems needing both token-level guarantees and broader graph awareness. The 40-relation schema can be extended to Freebase’s full relation set (approximately 24,000 relations). The formal framework for constraint quality enables systematic comparison of oracles across the hierarchy.

Reproducibility. All code is available at github.com/bernard/graph-constrained-reasoning. Experiments ran on a single RTX 4090 GPU with 16-bit precision. The Lean 4 proof is in [proof/formal_proof/](#).

References

- [2] Kurt Bollacker, Colin Evans, Praveen Paritosh, Tim Sturge, and Jamie Taylor. Freebase: A collaboratively created graph database for structuring human knowledge. In *Proceedings of*

- the 2008 ACM SIGMOD International Conference on Management of Data, pages 1247–1250, 2008.
- [4] Nicola De Cao, Gautier Izacard, Sebastian Riedel, and Fabio Petroni. Autoregressive entity retrieval. In *International Conference on Learning Representations*, 2021.
 - [6] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured NLP tasks without finetuning. In *The 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. URL <https://openreview.net/forum?id=KkHY1WGDII>.
 - [7] Gaole He, Yunshi Lan, Jing Jiang, Wayne Xin Zhao, and Ji-Rong Wen. Improving multi-hop knowledge base question answering by learning intermediate supervision signals. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining (WSDM)*, pages 553–561. ACM, 2021.
 - [8] Chen Huang, Xiusi Chen, Junzhou Zhang, Heng Li, and Yu Zheng. Think reverse and reason: Ontology-guided reverse thinking for knowledge graph reasoning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 17824–17838, 2025.
 - [9] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv preprint arXiv:2311.05232*, 2023.
 - [10] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Comput. Surv.*, 55(12), March 2023. ISSN 0360-0300. doi: 10.1145/3571730.
 - [13] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc., 2020.
 - [14] Kun Li, Tianhua Zhang, Xixin Wu, Hongyin Luo, James R. Glass, and Helen M. Meng. Decoding on graphs: Faithful and sound reasoning on knowledge graphs through generation of well-formed chains. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 24349–24364. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.1186. URL <http://dx.doi.org/10.18653/v1/2025.acl-long.1186>.
 - [16] Linhao Luo, Zicheng Zhao, Gholamreza Haffari, Yuan-Fang Li, Chen Gong, and Shirui Pan. Graph-constrained reasoning: Faithful reasoning on knowledge graphs with large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 41540–41565. PMLR, 2025. URL <https://proceedings.mlr.press/v267/luo25t.html>.
 - [18] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3982–3992. Association for Computational Linguistics, 2019.

- [19] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. Picard: Parsing incrementally for constrained auto-regressive decoding from language models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2021.
- [20] Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel M. Ni, Heung-Yeung Shum, and Jian Guo. Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. In *International Conference on Learning Representations*, 2024.
- [24] Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models, 2023.
- [27] Susumu Yata et al. marisa-trie: Matching algorithm with recursively implemented storage. <https://github.com/s-yata/marisa-trie>, 2024.
- [29] Bo Yuan, Hexuan Deng, Xuebo Liu, and Min Zhang. Routerkgqa: Specialized-general model routing for constraint-aware knowledge graph question answering, 2026.